DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING
Course Code and Title: CS-327 Computer Communication Networks
Complex Engineering Problem
Fall Semester 2026
TERM PROJECT

Project Title: Design, Implementation and Evaluation of a Multi-Host Networked Application

---

1. Course Learning Outcomes

- **CLO-1:** Discuss basic computer network topologies and reference models. (Taxonomy level C2, PLO-1 Engineering Knowledge)
- **CLO-2:** Explore operation and design issues of OSI layers. (Taxonomy level C3, PLO-2 Problem Analysis)
- **CLO-3:** Explore tools for environment friendly and/or sustainable communication. (Taxonomy level C3, PLO-7 Environment and Sustainability)

2. Complex Problem-Solving Attributes (CPA) covered (as per PEC OBA manual 2019)

- **CPA-1 Depth of analysis required:** Have no obvious solution and require abstract thinking, and originality in analysis to formulate suitable models.

  Addressed in this project by: no protocol specification is supplied. Each group must invent its own message formats, exchange sequences and error handling, then defend the design against measured evidence rather than against a model answer.

- **CPA-2 Level of interaction:** Require resolution of significant problems arising from interactions between wide-ranging or conflicting technical, engineering, or other issues.

  Addressed in this project by: Section 6 requires each group to identify and resolve at least three competing design tensions, such as reliability against latency or caching freshness against traffic reduction, where improving one measure demonstrably degrades another.

- **CPA-3 Familiarity:** Can extend beyond previous experiences by applying principles-based approaches.

  Addressed in this project by: students have so far studied protocols only as users of them. This project requires them to apply the layering, addressing and transport principles from Lectures 01 to 05 to build and operate a working distributed system across independent machines.

3. Problem Statement

Over the first five weeks of this course you have studied the Internet from the outside. You have seen how hosts attach to access networks, how packets are switched and delayed in the network core, how protocol layers encapsulate one another, and how an application layer protocol such as HTTP is structured. In every one of those cases you were a consumer of a protocol that somebody else designed.

In this Complex Engineering Problem, you will design one yourself. Your group will specify an original application layer protocol, implement both sides of it directly on the socket interface, deploy it across

several independent machines, capture its traffic on the wire, and measure how it behaves under real network conditions.

There is no single correct solution. Two groups building the same class of application will arrive at different protocols, and both may be defensible. Your marks depend on the quality of your engineering reasoning, the evidence you gather, and the honesty of your evaluation, not on matching a model answer.

4. Mandatory Requirements

All seven requirements below apply to every group, whichever application you select in Section 5.

R1. Design your own application layer protocol

Your protocol must be your own. In the report you must define the message types your protocol carries, the exact format of each message including every header field and its meaning, the order in which messages may legally be exchanged, the action each side takes on receipt of each message, and the status or error codes returned when something goes wrong. Write this section as a specification that a different group could implement from without ever seeing your code.

R2. Implement it directly on sockets

Implement the client and the server in any language you choose, such as Python, Java, C or C++. You must use the socket interface directly. You may not use a library that implements the application protocol for you. If you build a mail system you may not import an SMTP library, and if you build a web server you may not import a ready-made HTTP server framework. Helper libraries for unrelated concerns, such as file handling or plotting, are permitted.

R3. Deploy across multiple independent hosts

Your system must run on at least three independent network endpoints, each holding its own IP address. Any of the following arrangements is acceptable:

- Three physical machines connected through the department LAN, a lab switch or a mobile hotspot.
- Three virtual machines under VirtualBox, VMware or Hyper-V, configured in bridged or host-only networking so that each virtual machine receives a distinct IP address.
- A mixture of physical machines and virtual machines.

Running every component on 127.0.0.1 on a single laptop is not acceptable. A submission that cannot be demonstrated running across separate hosts will be assessed as an incomplete implementation regardless of the quality of the report. Your report must include the IP configuration of every host, captured from ipconfig or ip addr, and a network diagram of your deployment.

R4. Justify your transport layer choice

State whether each message type in your protocol travels over TCP or over UDP. Justify that choice against the four application service requirements covered in Lecture 04, namely reliable data transfer, throughput, timing and security. A justification that says only that TCP is reliable will earn no marks. Explain specifically what your application would lose if you had made the opposite choice.

R5. Capture your traffic and show encapsulation

Using Wireshark, capture your own protocol running on the link between two of your hosts. Include in the report an annotated screenshot of one captured packet in which you identify the link layer frame header, the network layer datagram header with its source and destination IP addresses, the transport layer

segment header with its source and destination port numbers, and your own application layer message carried inside the payload. Relate what you see to the encapsulation model studied in Lecture 03.

R6. Measure the performance of your system

Measure and report the round-trip time between your hosts, the application level throughput your system achieves, and the response or transfer time of one representative operation. Repeat every measurement at least five times and report the average together with the spread.

You must compare at least two conditions. Acceptable comparisons include a wired LAN against a wireless link, a cached response against an un cached one, a persistent connection against a new connection for every request, or small messages against large ones. Present the comparison as a table and as at least one graph, and explain the result using the delay model from Lecture 02, where the nodal delay is the sum of the processing, queueing, transmission and propagation delays.

R7. Demonstrate behaviour under failure

Show what your system does in at least one abnormal condition. Examples include the server being unreachable when a client starts, a client disconnecting in the middle of an exchange, two clients issuing conflicting requests at the same instant, or a UDP datagram being lost. Describe the behaviour you designed deliberately, and provide evidence from a log or a capture that it actually occurred.

5. Select One Application

Each group selects one of the following. Your selection is confirmed at the proposal stage.

- **Option A. Mini mail system.** A client submits a message to a mail server, the server stores it in the recipient mailbox, and a second client running on a different host retrieves it. A stronger submission runs two mail servers on separate hosts and relays mail between the two domains.
- **Option B. Mini DNS service.** An authoritative name server holds a zone of records. A resolver on a second host answers queries from clients on a third host and caches answers subject to a time to live. Demonstrate a cache hit and a cache miss, and measure the difference between them.
- **Option C. Mini web server with a caching proxy.** A server serves objects, a proxy on a second host caches them on behalf of clients on a third host. Implement status codes and both persistent and non-persistent connection modes, and measure the difference in page fetch time between the two modes.
- **Option D. Multi-client chat with presence.** A server relays messages between clients running on different hosts, maintains the list of users currently online, and notifies clients when a user joins or leaves. You must address message ordering when several clients send at the same time.
- **Option E. File distribution, client-server against peer to peer.** Distribute one file from a source to several receivers, first using a client-server scheme and then using a simple peer to peer scheme. Measure how the total distribution time changes as the number of receivers grows, and explain the trend.

6. Design Tensions You Must Resolve

Your report must contain a section in which you identify at least three of the following tensions, state the position you took, and defend it using your own measurements. This section carries significant weight because it is where engineering judgement is demonstrated.

- Reliability against latency. Retransmission protects your data but adds delay. Decide what your application actually needs.

- Connection reuse against server resources. Holding connections open shortens response time but consumes server memory and file descriptors.
- State against simplicity. HTTP is stateless because maintaining state is complex and the two views of that state may diverge after a crash. Decide whether your protocol keeps state, and account for what that decision cost you.
- Caching freshness against traffic reduction. A long time to live saves bandwidth but risks serving stale data.
- Compactness against readability. A binary message format is efficient on the wire, a text format is far easier to debug and extend.
- Security against performance, if you choose to add authentication or encryption.

7. Sustainability and Efficiency Analysis

Networked applications consume energy roughly in proportion to the traffic they generate and the work they impose on servers. Using the measurements you gathered for R6, estimate the reduction in bytes transferred that your design choices deliver. You might quantify the bytes saved by a cache hit compared with a cache miss, by connection reuse compared with repeated connection setup, or by a compact message format compared with a verbose one.

Express the saving as a percentage, project it to a realistic population of users, and comment on what that implies for bandwidth consumption and energy use at scale. One paragraph of honest arithmetic based on your own numbers is worth more than a page of general statements about green computing.

8. Deliverables

- **Project proposal (one-page, due end of Week 09).** Group members and roll numbers, the option selected, an outline of your intended protocol, the tools and languages you will use, your plan for obtaining three independent hosts, and the division of work among members.
- **Final report (maximum 10 pages, excluding the rubric title page, references and code appendix).** Structured according to Section 9.
- **Source code.** Submitted as an archive or a repository link. The code must build and run on the machines used in the demonstration.
- **Live demonstration (Week 13).** Your system will be started on the separate hosts at the time of submission. All group members must be present, and every member must be able to explain any part of the design, not only the part they wrote.

9. Required Report Structure

- Introduction and the option selected
- Protocol specification (R1)
- System and deployment architecture, with network diagram and IP addressing plan (R3)
- Implementation notes, including the transport layer justification (R2, R4)
- Packet capture and encapsulation analysis (R5)
- Performance measurement, results and graphs (R6)
- Design tensions and engineering justification (Section 6)
- Sustainability and efficiency analysis (Section 7)

- Behaviour under failure (R7)
- Individual contributions table, signed by all members
- References

10. Instructions

- There can be a maximum of 4 students in a group.
- The report must not exceed 10 pages, excluding the rubric title page, references and the code appendix.
- The report must include the network diagram, the IP addressing plan, the protocol message formats and the configuration of each host.
- Every group member must contribute to both the implementation and the report. The individual contributions table must be signed by all members.
- Any group that cannot demonstrate the system running across separate hosts will be assessed as an incomplete implementation, whatever the quality of the written report.
- Any use of AI assistance must be declared in an appendix stating what was used and for which part of the work.

11. Submission Guidelines

The project proposal must be submitted in person by all group members by the end of Week 09.

For the final report:

- Use font size 12 for body text and 14 for headings. The font style must be Times New Roman.
- Attach the provided rubric on top of your report, with names and roll numbers filled in. This serves as the title page, so no separate title page is required.
- Print and submit a hard copy of the report at the time of submission.
- All group members must be present during submission.
- For the online submission, save the report as a PDF using the naming convention CS24XXX.pdf, where XXX is the student roll number, for example CS24013.pdf.
- All group members must submit from their own accounts.
- Submit the project in person and upload all relevant files and the report to Google Classroom by the end of Week 13.
- Exact dates shall be communicated later.